

1. Deep-Dive Mathematical Derivation from First Principles

In Nomura's Cash Equities Central Risk Book (CRB) department, accurately classifying market regimes is a core component of managing overnight and execution risk. An Executive Director will want to know if you can translate algorithmic properties into quantitative risk metrics.

While a Hidden Markov Model (HMM) acts as a *generative sequential model*—learning the joint probability distribution $p(x, y)$ via state transitions—Gradient Boosting Machines (GBMs) act as a *discriminative conditional model*, learning the conditional probability $p(y \mid x)$ directly.

When training a discriminative model on rare events (like a crisis regime occurring on $\sim 5\%$ of trading days), standard optimization engines fail because they treat all misclassifications equally. To resolve this imbalance, we can use synthetic oversampling or alter the internal optimization engine via cost-sensitive updates.

Phase A: Line-by-Line Mechanics of SMOTE Neighborhood Interpolation

Synthetic Minority Over-sampling Technique (SMOTE) generates synthetic training vectors within the feature space to prevent a discriminative boundary from collapsing into a majority class. Let $\mathcal{X}_{\min} \subset \mathbb{R}^d$ be the subset of minority (crisis) feature instances.

Step 1: Metric Distance Selection

For each minority sample vector $\mathbf{x}_i \in \mathcal{X}_{\min}$, we compute the distance to all other minority observations $\mathbf{x}_m \in \mathcal{X}_{\min}$ using the Minkowski metric (typically the Euclidean distance, $p = 2$):

$$d(\mathbf{x}_i, \mathbf{x}_m) = \left(\sum_{j=1}^d |x_{i,j} - x_{m,j}|^p \right)^{\frac{1}{p}}$$

The k -nearest neighbors are defined as the set $\mathcal{N}_k(\mathbf{x}_i)$ containing the k samples with the smallest distance values $d(\mathbf{x}_i, \mathbf{x}_m)$.

Step 2: Linear Convex Combination Interpolation

To create a synthetic instance $\tilde{\mathbf{x}}$, the algorithm randomly selects one neighbor $\mathbf{x}_{nn} \in \mathcal{N}_k(\mathbf{x}_i)$ and applies a random uniform scaling parameter $\lambda \sim \mathcal{U}(0, 1)$:

$$\tilde{\mathbf{x}} = \mathbf{x}_i + \lambda (\mathbf{x}_{nn} - \mathbf{x}_i)$$

Step 3: Component-by-Component Line Expansion

To understand why this point remains within the local minority cluster, we can expand the vector equation line-by-line for each feature component $j \in \{1, \dots, d\}$:

$$\tilde{x}_j = x_{i,j} + \lambda (x_{nn,j} - x_{i,j})$$

Distributing the scalar λ :

$$\tilde{x}_j = x_{i,j} + \lambda x_{nn,j} - \lambda x_{i,j}$$

Gathering the terms for the anchor sample $x_{i,j}$:

$$\tilde{x}_j = (1 - \lambda)x_{i,j} + \lambda x_{nn,j}$$

Because $\lambda \in [0, 1]$, the coefficients $(1 - \lambda)$ and λ are non-negative and sum to 1:

$$(1 - \lambda) + \lambda = 1$$

This proves that $\tilde{\mathbf{x}}$ is a strict **convex combination** of $\mathbf{x}_i$ and $\mathbf{x}_{nn}$. Geometrically, this forces the synthetic instance to lie exactly on the line segment connecting the two real minority samples in $\mathbb{R}^d$, expanding the minority decision region without introducing arbitrary out-of-bounds noise.

Phase B: Cost-Sensitive Loss Function Transformations

Instead of altering the feature space, a more robust quantitative approach modifies the loss function during tree splitting and leaf weight determination. Let $y_t \in \{0, 1\}$ be the true regime label (1 = crisis, 0 = normal) and $p_t = \sigma(\hat{y}_t)$ be the predicted probability from the model's raw margin score $\hat{y}_t$, passed through the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$.

Step 1: Complete Formulation of Cost-Weighted Binary Cross-Entropy

We assign an explicit penalty multiplier w_1 to false negatives (missing a crisis) and w_0 to false positives. The loss function $\mathcal{L}$ over the entire training set is written as:

$$\mathcal{L} = - \sum_{t=1}^T [w_1 y_t \ln(p_t) + w_0 (1 - y_t) \ln(1 - p_t)]$$

In standard implementations like XGBoost, we normalize by setting $w_0 = 1$ and parameterizing $w_1 = \omega$, where $\omega = \frac{\text{Count}(y=0)}{\text{Count}(y=1)}$. This scale factor is often implemented via the `scale_pos_weight` parameter:

$$\mathcal{L} = - \sum_{t=1}^T [\omega y_t \ln(p_t) + (1 - y_t) \ln(1 - p_t)]$$

Step 2: Line-by-Line First Derivative (Gradient g_t)

To optimize the trees, a GBM requires the first derivative of the loss function with respect to the raw model projection $\hat{y}_t$. Using the chain rule:

$$g_t = \frac{\partial \mathcal{L}_t}{\partial \hat{y}_t} = \frac{\partial \mathcal{L}_t}{\partial p_t} \cdot \frac{\partial p_t}{\partial \hat{y}_t}$$

First, compute the derivative of the single-sample loss component $\mathcal{L}_t$ with respect to p_t :

$$\frac{\partial \mathcal{L}_t}{\partial p_t} = - \left[\frac{\omega y_t}{p_t} - \frac{1 - y_t}{1 - p_t} \right] = - \frac{\omega y_t (1 - p_t) - (1 - y_t) p_t}{p_t (1 - p_t)}$$

Next, compute the derivative of the sigmoid linkage function $p_t = \sigma(\hat{y}_t)$ with respect to $\hat{y}_t$:

$$\frac{\partial p_t}{\partial \hat{y}_t} = p_t (1 - p_t)$$

Combine the two derivatives to solve for the gradient g_t :

$$g_t = -\frac{\omega y_t(1 - p_t) - (1 - y_t)p_t}{p_t(1 - p_t)} \cdot p_t(1 - p_t)$$

Canceling out the denominator term $p_t(1 - p_t)$:

$$g_t = -[\omega y_t(1 - p_t) - (1 - y_t)p_t]$$

$$g_t = -\omega y_t + \omega y_t p_t + p_t - y_t p_t$$

Grouping terms by the predicted probability p_t :

$$g_t = p_t(\omega y_t + 1 - y_t) - \omega y_t$$

$$g_t = p_t(1 + (\omega - 1)y_t) - \omega y_t$$

- Evaluate for a True Positive/False Negative ($y_t = 1$): $g_t = p_t(1 + \omega - 1) - \omega = \omega(p_t - 1)$
- Evaluate for a True Negative/False

Positive ($y_t = 0$): $g_t = p_t(1 + 0) - 0 = p_t$

Combining these back into a single conditional indicator function:

$$g_t = \omega(p_t - 1)\mathbb{I}_{\{y_t=1\}} + p_t\mathbb{I}_{\{y_t=0\}}$$

Step 3: Line-by-Line Second Derivative (Hessian h_t)

To determine structural leaf weights and split scores via Newton-Raphson optimization, we take the derivative of the gradient g_t with respect to $\hat{y}_t$:

$$h_t = \frac{\partial g_t}{\partial \hat{y}_t} = \frac{\partial g_t}{\partial p_t} \cdot \frac{\partial p_t}{\partial \hat{y}_t}$$

Differentiating $g_t = p_t(1 + (\omega - 1)y_t) - \omega y_t$ with respect to p_t :

$$\frac{\partial g_t}{\partial p_t} = 1 + (\omega - 1)y_t$$

Since $\frac{\partial p_t}{\partial \hat{y}_t} = p_t(1 - p_t)$, substituting this back yields our final Hessian equation:

$$h_t = [1 + (\omega - 1)y_t] p_t(1 - p_t)$$

- Evaluate for a Crisis State ($y_t = 1$): $h_t = \omega p_t(1 - p_t)$
- Evaluate for a Normal State ($y_t = 0$): $h_t = p_t(1 - p_t)$

Combining these into an indicator function:

$$h_t = [\omega\mathbb{I}_{\{y_t=1\}} + \mathbb{I}_{\{y_t=0\}}] p_t(1 - p_t)$$

Step 4: Leaf Weight Optimization Equation

During tree expansion, a leaf node J groups a subset of samples I_J . The optimal leaf weight value w_J^* minimizes the local loss function. This is found via the Newton-Raphson step:

$$w_J^* = -\frac{\sum_{t \in I_J} g_t}{\sum_{t \in I_J} h_t} = \frac{\sum_{t \in I_J} [\omega(1 - p_t)\mathbb{I}_{\{y_t=1\}} - p_t\mathbb{I}_{\{y_t=0\}}]}{\sum_{t \in I_J} [\omega\mathbb{I}_{\{y_t=1\}} + \mathbb{I}_{\{y_t=0\}}] p_t(1 - p_t)}$$

This line-by-line derivation demonstrates the mathematical mechanics of `scale_pos_weight`. When a crisis event ($y_t = 1$) is encountered, both its gradient profile and its Hessian weight are amplified by ω . This prevents minority samples from being washed out by the majority class, forcing the tree structure to adjust its splitting decisions to capture the rare regime.

2. Quantitative Infrastructure & Application of Theory

The following production-grade script provides a full implementation of a custom Gradient Boosting engine optimized for imbalanced classification. It extracts macro and microstructural features, runs SMOTE sampling benchmarks, builds custom cost-sensitive loss handlers from scratch using `numpy` and `scipy.optimize`, and provides an interactive comparison dashboard using Plotly.

```
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.special import expit # Exact sigmoid calculation logic
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# =====
# 1. EXTRACT MACRO AND MICROSTRUCTURAL FEATURES
# =====
def generate_market_features_and_regimes(n_days: int = 1500, seed: int = 42) ->
pd.DataFrame:
    """
    Simulates cross-asset macro and microstructural features for a CRB.
    Enforces a realistic ~5% rare crisis regime occurrence profile.
    """
    np.random.seed(seed)

    # Base chronological timeline tracking array
    timeline = pd.date_range(start="2020-01-01", periods=n_days, freq='B')

    # Latent systemic stress variable generation (Autoregressive AR(1) proxy)
    latent_stress = np.zeros(n_days)
    for t in range(1, n_days):
        latent_stress[t] = 0.88 * latent_stress[t-1] + np.random.normal(0, 1)

    # Threshold latent stress to create a hard ~5% rare crisis class flag
    crisis_threshold = np.quantile(latent_stress, 0.95)
    crisis_labels = (latent_stress >= crisis_threshold).astype(int)

    # Synthesize Cross-Asset Signals mapped to latent states
    vix_base = 15.0
    vix = vix_base + np.exp(0.5 * latent_stress) * 5.0 + np.random.normal(0, 1,
n_days)
```

```

# Vol Premium: Realized Vol / Implied Vol (depressed during structural crisis)
vol_premium = 1.1 - 0.05 * latent_stress + np.random.normal(0, 0.05, n_days)

# Credit Spread: High Yield minus Investment Grade yield metrics
credit_spread = 1.5 + 0.4 * latent_stress + np.random.normal(0, 0.1, n_days)

# Market Microstructure: Bid-Ask spread widening during systemic corrections
bid_ask_spread = 0.01 + 0.005 * latent_stress + np.random.normal(0, 0.001,
n_days)

df = pd.DataFrame({
    'VIX_Level': vix,
    'Vol_Premium': vol_premium,
    'Credit_Spread': credit_spread,
    'Bid_Ask_Spread': bid_ask_spread,
    'Crisis_Regime_Target': crisis_labels
}, index=timeline)

return df

# =====
# 2. VECTORIZED SMOTE ENGINE FROM FIRST PRINCIPLES
# =====
def execute_vectorized_smote(X: np.ndarray, y: np.ndarray, k_neighbors: int = 5,
oversample_ratio: float = 1.0) -> tuple:
    """
    Vectorized implementation of SMOTE to balance minority classes.
    Avoids external library dependencies to match custom core engine requirements.
    """
    minority_mask = (y == 1)
    X_min = X[minority_mask]

    if len(X_min) == 0:
        return X, y

    # Calculate pairwise Euclidean distance matrix across the feature space
    distance_matrix = cdist(X_min, X_min, metric='euclidean')

    # Sort indexes along axis to extract spatial neighborhoods
    neighbor_indexes = np.argsort(distance_matrix, axis=1)[: , 1:k_neighbors+1]

    n_synthetic_required = int(len(X_min) * oversample_ratio)
    synthetic_samples = np.zeros((n_synthetic_required, X.shape))

    for i in range(n_synthetic_required):
        # Pick random anchor sample from minority pool
        anchor_idx = np.random.randint(0, len(X_min))
        # Select a random neighbor index from its k-nearest neighborhood
        chosen_neighbor_col = np.random.randint(0, k_neighbors)
        neighbor_idx = neighbor_indexes[anchor_idx, chosen_neighbor_col]

        # Pull feature vectors
        x_anchor = X_min[anchor_idx]
        x_neighbor = X_min[neighbor_idx]

```

```

        # Enforce convex combination interpolation identity step
        lambda_param = np.random.uniform(0, 1)
        synthetic_samples[i, :] = x_anchor + lambda_param * (x_neighbor -
x_anchor)

    # Combine synthetic and original data
    X_balanced = np.vstack([X, synthetic_samples])
    y_balanced = np.concatenate([y, np.ones(n_synthetic_required, dtype=int)])

    return X_balanced, y_balanced

# =====
# 3. COST-SENSITIVE OPTIMIZATION LOGISTIC ENGINE
# =====
class CostSensitiveLogisticBooster:
    """
    Custom classifier optimizing a cost-sensitive objective.
    Explicitly tracks custom gradient and Hessian vectors to handle class
    imbalance.
    """
    def __init__(self, scale_pos_weight: float = 1.0, learning_rate: float = 0.1,
n_estimators: int = 10):
        self.omega = scale_pos_weight
        self.lr = learning_rate
        self.n_estimators = n_estimators
        self.base_pred = 0.0
        self.weights = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        T, d = X.shape
        # Initialize raw margins to zero
        raw_margins = np.zeros(T)
        self.weights = np.zeros(d)

        # Simple gradient ascent loop as a clear programmatic proxy for a custom
        tree model
        for _ in range(self.n_estimators):
            p = expit(raw_margins)

            # Analytical Gradient Vector Equation derivation check:  $g = p(1 + (w-1)y) - w*y$ 
            g = p * (1.0 + (self.omega - 1.0) * y) - self.omega * y
            # Analytical Hessian Vector Equation derivation check:  $h = (1 + (w-1)y) * p * (1 - p)$ 
            h = (1.0 + (self.omega - 1.0) * y) * p * (1.0 - p)
            h = np.clip(h, 1e-6, None) # Numerical stabilization block

            # Calculate optimal Newton step across feature weights
            for j in range(d):
                X_col = X[:, j]
                g_j = np.sum(g * X_col)
                h_j = np.sum(h * (X_col ** 2))

```

```

        # Update parameter arrays
        delta_w = - (g_j / (h_j + 1e-8))
        self.weights[j] += self.lr * delta_w

    # Refresh raw margin calculations
    raw_margins = X @ self.weights

def predict_proba(self, X: np.ndarray) -> np.ndarray:
    """Transforms margins into calibrated probabilities."""
    return expit(X @ self.weights)

# =====
# 4. INTERACTIVE PLOTLY INTERVIEW DASHBOARD CANVAS
# =====
def generate_imbalance_dashboard(df: pd.DataFrame, orig_X, orig_y, smote_X,
                                smote_y, p_naive, p_cost):
    """
    Renders an interactive high-fidelity 2x2 multi-panel grid
    comparing SMOTE augmentation paths against Custom Cost-Sensitive Loss
    optimizations.
    """
    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=(
            "Original Imbalanced Data Distribution (VIX vs Credit Spread)",
            "Balanced SMOTE Augmented Feature Vector Topography",
            "Naive Loss Optimization Model Confidence Over Timeline",
            "Custom Cost-Sensitive Loss Model Calibrated Probability Output"
        ),
        vertical_spacing=0.15, horizontal_spacing=0.12
    )

    # Panel 1: Naive Distribution
    fig.add_trace(go.Scatter(
        x=df['Credit_Spread'], y=df['VIX_Level'], mode='markers',
        marker=dict(color=orig_y, colorscale=['cyan', 'crimson'], size=5),
        name='Original Space'
    ), row=1, col=1)

    # Panel 2: SMOTE Distribution
    fig.add_trace(go.Scatter(
        x=smote_X[:, 2], y=smote_X[:, 0], mode='markers',
        marker=dict(color=smote_y, colorscale=['cyan', 'crimson'], size=5),
        name='SMOTE Balanced Space'
    ), row=1, col=2)

    # Panel 3: Naive Output Probability Timeline
    time_axis = df.index
    fig.add_trace(go.Scatter(x=time_axis, y=p_naive, mode='lines',
        line=dict(color='orchid', width=1.5), name='Naive Prob'), row=2, col=1)
    # Overlay horizontal classification cut-off threshold boundary line
    fig.add_trace(go.Scatter(x=time_axis, y=[0.5]*len(time_axis), mode='lines',
        line=dict(color='white', dash='dash'), name='Threshold'), row=2, col=1)

```

```

# Panel 4: Cost Sensitive Calibrated Outputs
fig.add_trace(go.Scatter(x=time_axis, y=p_cost, mode='lines',
line=dict(color='gold', width=1.5), name='Cost Sensitive Prob'), row=2, col=2)
fig.add_trace(go.Scatter(x=time_axis, y=[0.5]*len(time_axis), mode='lines',
line=dict(color='white', dash='dash')), row=2, col=2)

fig.update_layout(
    title_text="Nomura Central Risk Book – Regime Boost Optimization Platform
Context Canvas",
    title_x=0.5, height=900, width=1300, template="plotly_dark",
    showlegend=False
)

fig.update_xaxes(title_text="Credit Spread Score Indicator", row=1, col=1)
fig.update_yaxes(title_text="VIX Realized Pricing Matrix", row=1, col=1)
fig.update_xaxes(title_text="Credit Spread Score Indicator (Augmented)",
row=1, col=2)
fig.update_yaxes(title_text="VIX Realized Pricing Matrix (Augmented)", row=1,
col=2)
fig.update_xaxes(title_text="Chronological Session Timeline", row=2, col=1)
fig.update_yaxes(title_text="Assigned Crisis Likelihood P(y=1)", row=2, col=1)
fig.update_xaxes(title_text="Chronological Session Timeline", row=2, col=2)
fig.update_yaxes(title_text="Assigned Crisis Likelihood P(y=1)", row=2, col=2)

fig.show()

if __name__ == "__main__":
    # 1. Pipeline microstructural features matrix
    market_df = generate_market_features_and_regimes()

    X_features = market_df[['VIX_Level', 'Vol_Premium', 'Credit_Spread',
'Bid_Ask_Spread']].values
    y_target = market_df['Crisis_Regime_Target'].values

    # 2. Balance the features matrix using our custom SMOTE engine
    smote_X_mat, smote_y_vec = execute_vectorized_smote(X_features, y_target,
k_neighbors=5, oversample_ratio=0.8)

    # 3. Calibrate alternative boosting configurations
    # Model A: Naive baseline setup matching standard cross-entropy assumptions
    naive_booster = CostSensitiveLogisticBooster(scale_pos_weight=1.0,
learning_rate=0.05, n_estimators=15)
    naive_booster.fit(X_features, y_target)
    probs_naive = naive_booster.predict_proba(X_features)

    # Model B: Custom cost-sensitive booster using our derived gradient/Hessian
scaling factor
    imbalance_weight = float((len(y_target) - np.sum(y_target)) /
np.sum(y_target))
    cost_booster = CostSensitiveLogisticBooster(scale_pos_weight=imbalance_weight,
learning_rate=0.05, n_estimators=15)
    cost_booster.fit(X_features, y_target)
    probs_cost = cost_booster.predict_proba(X_features)

```



```

print("\n" + "="*65)
print("CRB RISK CLASSIFIER CALIBRATION MATRIX:")
print("="*65)
print(f"Total Operational Market Horizon Days : {len(y_target)}")
print(f"Observed Crisis Sample Occurrences      : {np.sum(y_target)} (~
{np.mean(y_target)*100:.2f}% of data)")
print(f"Calculated Custom scale_pos_weight      : {imbalance_weight:.4f}")
print(f"Naive Model Detected Crisis Counts       : {np.sum(probs_naive >= 0.5)}")
print(f"Cost-Sensitive Detected Crisis Counts    : {np.sum(probs_cost >= 0.5)}")
print("="*65 + "\n")

# 4. Render Validation Platform Dashboard
generate_imbalance_dashboard(market_df, X_features, y_target, smote_X_mat,
smote_y_vec, probs_naive, probs_cost)

```

3. Strategic Interview Follow-Ups & Edge-Case Vulnerabilities

Follow-Up 1: Dangers of SMOTE Interpolation on High-Frequency Time Series

Question: "You explained how SMOTE creates synthetic instances using convex combinations. If we apply standard SMOTE directly to highly auto-correlated time-series features (like raw asset volatility or daily credit spreads), what risk do we introduce to our validation results? How does it affect our risk management boundary?"

- **The Answer:** Standard SMOTE assumes observations are independent and identically distributed (*i.i.d.*), ignoring temporal dependencies and autocorrelation structures.
- **The Structural Risk:** If we interpolate between a crisis observation at time t and another at time $t + k$ across the feature space, the synthetic point breaks the historical context. It separates the feature combinations from their true temporal dependencies, creating unrealistic, decoupled feature states.
- **The Consequence for the Book:** This populates the training matrix with synthetic trading days that violate microstructural realities (e.g., a high VIX level combined with an overnight narrowing of the bid-ask spread). Training a discriminative classifier on these non-physical data points broadens the decision boundary artificially, leading to false positives that misallocate capital within the CRB.

Follow-Up 2: Why Precision-Recall AUC Trumps ROC AUC for Rare Regime Risk

Question: "When evaluating our regime classifiers under severe class imbalance (e.g., 5% crisis days), you stated that we should use Precision-Recall AUC (PR-AUC) rather than standard Receiver Operating Characteristic AUC (ROC-AUC). Prove why ROC-AUC can provide a falsely optimistic view of a model's performance when managing a portfolio."

- **The Answer:** This comes down to how the two metrics handle the majority class (True Negatives).
- **The Mathematical Defect of ROC:** The ROC curve plots the True Positive Rate against the False Gamma/Positive Rate (FPR). The formula for FPR is:

$$\text{FPR} = \frac{\text{False Positives}}{\text{False Positives} + \text{True Negatives}}$$

- In a CRB dataset with 10,000 trading days, 9,500 are normal ($y = 0$). If our model predicts poorly and generates 500 False Positives, the denominator includes all 9,500 True Negatives:

$$\text{FPR} = \frac{500}{500 + 9500} = \frac{500}{10000} = 0.05$$

- An FPR of only 5% keeps the ROC curve close to the upper-left corner, giving a deceptively high ROC-AUC score (e.g., > 0.90) despite the high number of false alarms.
- **The Solution via PR-AUC:** Precision isolates the model's performance relative to the positive predictions, removing the large True Negative count from its denominator:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- If the model correctly identifies 250 out of 500 crisis days but generates 500 False Positives, the true precision is:

$$\text{Precision} = \frac{250}{250 + 500} = \frac{250}{750} = 33.3\%$$

- PR-AUC captures this performance drop directly. For a risk book manager, a precision of 33% means 2 out of 3 risk signals are false alarms, which would lead to unnecessary and costly defensive hedging. PR-AUC reveals this risk clearly, whereas ROC-AUC hides it.